

One filter, many models

A plain-language explanation of the macro idea, with SEIR shown both ways · [PhyloPOMP.jl](#)

The idea in one sentence. Write the fiddly filter math *once*, describe each model as a *short list of events*, and let a macro glue the two together — so a term can be forgotten in one place at most, and checked in one place at most.

1 · What's going on today

Every model — SEIR, MERS, BDEI — needs a **filter**: the code that scores how well that model (with given parameters) explains an observed viral family tree. That scoring math is long and delicate. At each branch and each sample it multiplies in small correction factors: logs, ratios, and counts of how many tree lineages sit in each compartment.

The catch is that **each model carries its own hand-written copy of that math**. When you copy long math by hand and adapt it for a new model, terms get dropped. That is precisely what the MERS audit kept finding: a missing $+\log(\ell)$ on a cross-species swap, a missing $+\log(2)$ on a cross-species branch split, two birth rates transposed. Small slips — the kind that only happen because the same delicate math is retyped per model.

2 · The picture behind the fix

In a kitchen	In the code
A recipe card — ingredients and steps	A model — its list of events
The cooking technique	The filter math (the likelihood)
One master chef who knows the technique	One shared filter , written once
A blank recipe-card template	A macro (a template that writes code for you)

Today is like making every cook re-learn the whole technique for every recipe, and hoping none of them forgets the salt. The fix: one master chef who knows the technique; each recipe is just a short card; a standard template makes the cards quick and uniform. Nobody re-derives the math, so nobody drops a term.

What a "macro" actually is

A macro is just a **fill-in-the-blank template that writes code for you**. You type something short; it expands into the longer code you'd otherwise write by hand:

```
You write:      @greet Alice      →      The macro expands it into:
println("Hello, Alice!")
```

It does **not** do the filter math. It only stamps out the boring model description. The shared filter still does the math. Keeping those two jobs apart is the whole point.

3 · The SEIR model

The whole model, written as one form

Here is the entire SEIR model the way the macro would take it — one short block. This is the complete description; everything else is generated from it.

```
@mgp SEIR begin
  compartments = (S, E, I, R)      # the populations we track
  demes         = (E, I)          # the ones that can appear on the tree

  @event infection    rate = β*S*I/N    effect = (S→E)    tree = branch_splits
  @event progression  rate = σ*E        effect = (E→I)    tree = relabel
  @event recovery     rate = γ*I        effect = (I→R)    tree = lineage_ends
  @event waning       rate = ω*R        effect = (R→S)    tree = none
  @event sampling     rate = ψ*I        effect = observe  tree = tip
end
```

(Real sampling has two flavours — a plain sample and a sample-with-removal at rate χ . That split is shown in the sampling row of the comparison below; it's kept as one line here for readability.)

The same events, as a plain table

Event	How often (rate)	Effect on the population	Effect on the tree
Infection	$\beta \cdot S \cdot I / N$	one S becomes E	a branch splits
Progression	$\sigma \cdot E$	one E becomes I	a lineage is relabelled
Recovery	$\gamma \cdot I$	one I becomes R	a lineage ends
	$\omega \cdot R$		

Waning		one R becomes S	nothing
Sampling	$\psi \cdot I$	we observe an infected host	a tip appears

The same events, shown both ways

Below is every SEIR event. On the **left** is the one line you'd write with the macro. On the **right** is the real code that does the same event *today* — taken from `seir_naive.jl`, and note that a single event is spread across three different functions (`event_rates!`, `regular_part!`, `singular_part!`), each carrying its own hand-typed correction terms.

With the macro — you write this	Today — you write (and must audit) this
Infection <code>@event infection rate = β*S*I/N effect = (S-E) tree = branch_splits</code>	<code>IN EVENT_RATES! alpha[1]=alpha[2]= β*S*I/pop pi[1]=@indicator(I>0, 1-ellI/I) pi[2]=@indicator(I>0, ellI/I) IN REGULAR_PART! (BACKGROUND) k==1: S-=1; E+=1; ll += log(1-ellE/E) k==2: ll += log(ellI); swap!(cols,Infec,Expos,b) S-=1; E+=1; ll += log(1-ellI/I)-log(E) IN SINGULAR_PART! (A BRANCH ON THE TREE) ll += log(β*S*I/pop); fork!(cols,Infec,...) S-=1; E+=1; ll -= log(E*I)</code>
Progression <code>@event progression rate = σ*E effect = (E-I) tree = relabel</code>	<code>IN EVENT_RATES! alpha[3]=alpha[4]= σ*E pi[3]=@indicator(E>0, 1-ellE/E) pi[4]=@indicator(E>0, ellE/E) IN REGULAR_PART! (BACKGROUND) k==3: E-=1; I+=1; ll += log(1-ellI/I) k==4: ll += log(ellE); swap!(cols,Expos,Infec,b) E-=1; I+=1; ll -= log(I)</code>
Recovery <code>@event recovery rate = γ*I effect = (I-R) tree = lineage_ends</code>	<code>IN EVENT_RATES! (RATE + A DECAY TERM) alpha[5]=@indicator(I>ellI, γ*(I-ellI)) ... γ*ellI + @indicator(IseII, γ*(I-ellI)) IN REGULAR_PART! k==5: ll -= log(1-ellI/I); I-=1; R+=1</code>
Waning <code>@event waning rate = ω*R effect = (R-S) tree = none</code>	<code>IN EVENT_RATES! + REGULAR_PART! alpha[6]= ω*R k==6: R-=1; S+=1</code>
Sampling <code>@event sampling rate = ψ*I effect = observe tree = tip</code>	<code>IN SINGULAR_PART! (A TIP ON THE TREE) k,_,p = rcateg([ψ,χ], true); ll -= log(p) chop!(cols,Infec,...) k==1: ll += log(ψ*I) + log(1-ellI/I) # plain sample k==2: ll += log(χ*I); I-=1 # sample + removal</code>

Read one row and the point is visible: **one readable line on the left; a dozen scattered lines with hand-typed `log(...)` corrections on the right**. Those correction terms are exactly what goes missing when the math is copied per model.

Where your MERS bugs live in this picture. Look at the highlighted `ll += log(ellI)` on the infection swap. SEIR happens to get it right. The MERS audit found that the *same class of term* — the `+log( $\ell$ )` owed to a cross-species swap — had been dropped. In the shared-filter design that term is written and verified once, inside the swap operation itself, so no model can be missing it.

4 · What adding a new model looks like

You write another short form — for MERS: camels, humans, spillover, sampling — and stop. You do **not** touch the filter math. The macro stamps out the plumbing; the shared filter reads it and computes the likelihood. Running `@macroexpand` shows you the exact event table that got generated, which is the thing you diff against the King-Lin-Ionides event construction — turning your audit's event-table step into a quick comparison instead of a re-derivation.

5 · What this fixes, and what it doesn't

- Fixes:**
- A dropped bookkeeping term (`+log( $\ell$ )`, `+log(2)`, a binomial ratio) can now only happen in the one shared function — not in every branch of every model.
 - A swapped rate (camel vs. human) is a single labelled line in the form, where a transposition is obvious to the eye, instead of buried in indexed code.
 - "Is this event regular or singular?" becomes a labelled field, not something re-decided per model.

Does not fix (still your job): the shared filter math still has to be *correct*. Someone still verifies it once, carefully, against the paper. The design doesn't remove that work — it means you do it **once, for all models**, instead of redoing and re-auditing it every time a new model appears.

Summary. Short form per model + one shared filter written once + a macro to connect them. Fewer places to make a mistake; one place to check it.